

# Response to the Editors and Reviewers

Manuscript DAJOUR-D-26-00332

“An Interpretable Temporal Graph Learning Method for Real-Time Financial Fraud Detection and Decision Making”

(revised title: “A Self-Explaining Temporal Graph Learning Method for Low-Latency Financial Fraud Detection and Decision Support”)

Dear Professor Tavana, dear Associate Editor, dear Reviewers,

Thank you for the careful and candid evaluation of our manuscript and for the opportunity to revise it. The reviewers’ central concern, shared by the Associate Editor, was that the claims of the manuscript were not supported by the evidence. We agree with this assessment. To address it, we extended the implementation as requested (new baselines, redesigned objective, new analyses), re-ran all experiments under a single documented protocol with multiple seeds, and revised every claim so that it is supported by the reported evidence. Section and table numbers below refer to the revised manuscript. All numbers in the manuscript and in this letter are generated automatically from the per-run records released with the code, which also addresses the consistency concerns raised by both reviewers. Because most of the text was rewritten, the version with changes marked against the first version (`RTXGNN_revision1_marked.pdf`) highlights large parts of the manuscript.

## Summary of the main changes

- All experiments re-run under a unified protocol.** As requested by both reviewers, all results are now averages over 10 seeds with significance tests. All models share one code base, one preprocessing (quantile normalisation fitted on the training steps for neural models), one training budget and one threshold-selection rule (maximum F1 on the validation steps). The temporal split is unchanged (train 1–30, validation 31–34, test 35–49). Because the protocol, the objective and the set of baselines changed, the numbers differ from those of the first version. All code, per-run records and the scripts that generate the tables are public ([https://github.com/vinhqddang/explainable\\_GNN\\_financial\\_fraud](https://github.com/vinhqddang/explainable_GNN_financial_fraud)).
- Stronger and accurately positioned results.** Under the unified protocol RTXGNN reaches F1 0.700 and AP 0.771 (10 seeds), compared with F1 0.513 and AUC 0.866 in the first version. It matches the best GNN baseline (GAS, F1 0.701) and is ahead on average of all other graph models, including the self-explaining SEFraud (+0.054 F1), although most of these differences are within the variation across seeds. Gradient-boosted trees on the same features remain more accurate (XGBoost, F1 0.790), and we report this openly. The claims of state-of-the-art performance, of a “19.3% improvement” and of “production readiness” have been replaced by these measured statements.
- A redesigned method.** (i) The fidelity loss criticised by Reviewer 1 was replaced by sufficiency and necessity terms on *hard* top- $k$  explanations. (ii) The label-supervised temporal consistency loss was removed. (iii) HRAPE was made shift-invariant: it no longer encodes absolute time, which lowered and destabilised performance under the temporal split. (iv) The prediction loss was changed from focal loss to class-balanced cross-entropy. (v) Quantile normalisation of the features, fitted on the training steps only, is now applied for all neural models; it raises the validation AP of 5 of the 6 neural models tested ([Appendix D](#)). The pseudocode was rewritten to match the code line by line, and an inference algorithm was added.
- A much broader evaluation.** The evaluation now includes 16 baselines (adding LR, RF, XGBoost, TGAT, TGN, APAN, FRAUDRE and GAS), 10 seeds with Holm-corrected Wilcoxon

tests, and an operational analysis of alerts and misses. We added per-step results for steps 35–49 with class counts, a drift analysis with a retraining strategy, and stratified label-efficiency experiments with 5 subsets. The explanation evaluation now covers sufficiency and necessity at four explanation sizes, a random baseline, saliency, Integrated Gradients, GNNExplainer, SEFraud’s masks, class-wise results, stability and cost. We measure end-to-end CPU latency (p50/p95/p99, stage breakdown, throughput, memory) and report a second real-world financial dataset (T-Finance). An equal-budget hyperparameter search for RTXGNN and ten baselines, selected on validation data only, confirms that the conclusions do not depend on the hyperparameters. The synthetic benchmark was redesigned with ground-truth explanations and a fully documented generator.

5. **Careful wording.** “Intrinsically interpretable” was replaced by “self-explaining” throughout, the regulatory claims were rewritten (Section 6.2), and the case studies no longer assign meanings to anonymised features.

## Associate Editor and Editor-in-Chief

*Both reviewers find the topic relevant, but they raise serious and overlapping concerns, most notably that the manuscript’s claims are not adequately supported by the evidence presented.*

**Response.** We agree. The list above summarises how we addressed this. The detailed responses follow. Where a reviewer’s concern revealed a claim that the new evidence does not support, we removed or weakened the claim rather than defending it.

## Reviewer 1

*1.1 Reporting an F1-score of 0.5129 and calling it “strong predictive performance” [...] you picked the weakest competitor to calculate that percentage. An F1 of 0.51 means you are missing nearly half of all fraud cases [...] You need to be more upfront about what this score actually means.*

**Response.** We agree. The 19.3% figure was computed against GAT rather than against the strongest baseline, and “strong” was not justified for an F1 of 0.51. In the revision, RTXGNN obtains  $F1\ 0.700 \pm 0.045$  (precision 0.680, recall 0.728, AP 0.771) over 10 seeds, a substantial improvement over the first version, owing to the redesigned temporal encoding, the new objective and a better input normalisation. RTXGNN matches the best GNN baseline (GAS,  $F1\ 0.701$ ; difference not significant) and is ahead of all other GNNs on average, including SEFraud ( $+0.054\ F1$ ,  $p = 0.010$  uncorrected, not significant after Holm correction). XGBoost (0.790) is significantly more accurate, and the random forest reaches 0.733. These values are in the range reported in the literature for this split (e.g. Weber et al., 2019). We no longer use relative improvement figures. We now state plainly what the score means: at the validation-chosen threshold, RTXGNN misses 27% of the 1,083 illicit test transactions, and 32% of its alerts are false (Table 5, which reports alerts, misses and false alerts, as well as precision at fixed alert budgets). We also show that the ranking is what makes the model useful for decision support: the 500 highest-scored test transactions have a precision of 0.99. A model with these error rates should be used as a prioritisation aid in a human review process, not as an autonomous decision maker (Section 6.1).

**Changes in the manuscript.** Abstract, Introduction (“Summary of findings”), Sections 5.1, 5.3, 6.1, Conclusions; new Table 5.

*1.2 Baseline selection [...] Where is TGN? Where is APAN? What about TGAT? [...] FRAUDRE or GAS [...] It feels like you have cherry-picked baselines.*

**Response.** We added TGAT, TGN, APAN, FRAUDRE and GAS, and also logistic regression, random forest and XGBoost, which are known to be strong on Elliptic. The comparison now contains 16 baselines in four groups (Table 3): feature-only models, general GNNs, temporal GNNs

(EvolveGCN-O, TGAT, TGN, APAN) and fraud-specific GNNs (CARE-GNN, PC-GNN, GAS, FRAUDRE, SEFraud). All share the same protocol, budget and threshold selection, and all are run with 10 seeds. To rule out that the comparison favours RTXGNN through the choice of hyperparameters, we added an equal-budget tuning study (Section 5.2, Table 4). RTXGNN and ten baselines (the tree ensembles, GCN, GraphSAGE, GAT, TGAT, TGN, GAS, FRAUDRE and SEFraud) each received 20 configurations, selected by validation AP only and then evaluated with 10 seeds. Tuning improves several baselines, for example the random forest (+0.036 F1) and GAT (+0.065), and leaves the conclusions unchanged: the tree ensembles remain significantly more accurate, and RTXGNN (F1 0.703) is not significantly different from the tuned GNNs. Because the original implementations target other graph types (multi-relation review graphs, heterogeneous graphs, link prediction on continuous-time graphs), we re-implemented all graph baselines in one code base and document each adaptation (Appendix B). We also explain why continuous-time models have little to exploit on Elliptic: every transaction appears in a single snapshot and no edge crosses snapshots (Section 2). For the same reason DySAT reduces to a structural attention network, which is represented by GAT. H2-FDetector and DGA-GNN, which appeared in the first version, were not re-run under the new protocol: their central mechanisms (separating homophilic and heterophilic connections per relation, and decision-tree binning of raw attributes) are represented on Elliptic’s single-relation graph by CARE-GNN, PC-GNN, FRAUDRE and the tree ensembles. The first-version numbers of these models were obtained under a different protocol and are not comparable with the new results (Section 4.3). The “state-of-the-art” claim has been removed.

**Changes in the manuscript.** Sections 4.3, 5.1; Table 3; Appendix B.

*1.3 Your model is not truly interpretable [...] self-explainability at best [...] toning down this language.*

**Response.** We agree and have adopted the distinction proposed by the reviewer (and by Rudin, 2019). RTXGNN is now described as a *self-explaining* model whose masks are attributions computed in the forward pass. The terms “interpretable” and “intrinsically interpretable” were removed from the title and the text, and a terminology paragraph was added at the start of Section 3.

**Changes in the manuscript.** Title, Abstract, Section 3 (Terminology), Table 2.

*1.4 The fidelity loss [...] forcing the model to produce the same prediction on a masked graph as on the full graph [...] could easily result in masks that are completely meaningless.*

**Response.** The reviewer is right: with soft masks only, the original loss is minimised trivially by masks close to one or by a model that ignores the masks. We redesigned the objective (Section 3.4). The model is now evaluated on two *hard* views built from the top- $k$  masks with straight-through gradients: the explanation alone ( $k_f = 10$  of 165 features and  $k_e = 2$  incoming edges per node) and its complement. The sufficiency term requires the explanation alone to reproduce the prediction. The necessity term requires the complement to be uninformative, i.e. to push the prediction back to the class prior. We argue in the paragraph “Why the objective cannot be met by ignoring the masks” that neither all-ones masks nor mask-insensitive models minimise this objective. We then evaluate the explanations with the standard  $\text{Fid}^+/\text{Fid}^-$  metrics against post-hoc explainers applied to the *same* model and against a random explanation (Table 11). The sufficiency term has a clear effect: with it, keeping only the top-10 features reproduces the prediction within 0.076 ( $\text{Fid}^-$ ), against 0.233 for the same architecture trained without the two terms, and without a significant change in detection performance (Table 6). We also report the limits of the masks candidly. They are weakly *necessary*, because Elliptic’s features are redundant and the masks are nearly binary, keeping about half of the features open. GNNExplainer attains higher fidelity on the same model, but at about 300 times the cost and with much less stable explanations. Because the objective and the metrics both use perturbations, we discuss the resulting risk of favouring our method in the limitations.

**Changes in the manuscript.** Section 3.4; Tables 6 and 11; Figure 4; Section 6.5.

*1.5 The algorithm [...] variable naming is inconsistent, [...] broken array indexing [...] The edge mask loop runs over all edges, but you mention neighborhood sampling [...] Your pseudocode should reflect what you actually implemented.*

**Response.** Algorithm 1 was rewritten from the code and uses exactly the notation of Section 3. Training is full-batch on the subgraph induced by the 2-hop neighbourhoods of the labelled training nodes, which is exact for a 2-layer model, so no neighbourhood sampling is used during training. Neighbourhood sampling (fan-out 25 per hop) is used only at inference and is now described separately in Algorithm 2. Both algorithms name the source files that implement them. The implementation details in Section 3.5 now describe exactly the configuration used in all reported experiments. The algorithms are now placed in the method section, not after the references.

**Changes in the manuscript.** Section 3.5, Algorithms 1 and 2.

*1.6 The 76 percent performance retention with only 10 percent labeled data [...] You do not describe the sampling strategy at all [...] explain your stratified sampling approach and show that this result is robust and not just a lucky data split.*

**Response.** The first version subsampled the training labels uniformly at random, once. We now sample within every training time step and class, so each subset keeps the class ratio and the temporal coverage of the full training set. At least one illicit transaction is kept per step when the step has any. Each fraction is repeated with 5 different subsets, and subset  $r$  is combined with model seed  $r$ . We would like to clarify that the validation and test sets were never subsampled, so the test F1 is always computed on all 1,083 illicit test transactions. Table 10 reports mean  $\pm$  standard deviation for RTXGNN, random forest, GAT and MLP, together with the number of training labels (and illicit labels) at each fraction. With 10% of the labels, RTXGNN retains 87% of its full-label F1 (Section 5.7), with a standard deviation across subsets of 0.02, so the result does not depend on a lucky split.

**Changes in the manuscript.** Section 5.7, Table 10, Figure 3.

*1.7 Your discussion of GDPR and ECOA compliance feels like it was tacked on [...] a more concrete discussion of how your masks could be operationalized in practice.*

**Response.** We withdrew the statement that the masks “satisfy” GDPR Article 22 or ECOA and rewrote the discussion (Section 6.2). It now explains why an architecture cannot satisfy obligations that concern institutions and processes, notes that ECOA governs consumer credit rather than cryptocurrency transaction screening, and states that rankings of anonymous feature indices are not meaningful explanations. It then describes four concrete requirements for operationalising the masks: a governed feature dictionary, reason codes curated by compliance experts, human review with an audit trail (score, threshold, model version and attributions logged per alert, without a separate explanation step), and validation of the explanations themselves, including a user study that we have not yet conducted.

**Changes in the manuscript.** Sections 2 (Regulation) and 6.2; Conclusions.

*1.8 You are overinterpreting your ablation results [...] compare against a version that uses a simpler temporal encoding, like a basic time2vec.*

**Response.** We agree, and the new ablation (Table 6, 5 seeds) contains exactly this comparison. HRAPE is replaced by Time2Vec on the same time gaps, and additionally by absolute-time encodings (HRAPE with absolute time, and Time2Vec on absolute time). The “without HRAPE” variant keeps the SEAL masks and the explanation objective, so it isolates the temporal encoding. HRAPE is more accurate than Time2Vec (0.696 versus 0.629 F1; paired  $t$ -test over 5 seeds,  $p = 0.04$ , uncorrected, since a Wilcoxon test cannot reach  $p < 0.05$  with 5 pairs), and encoding absolute time lowers the mean F1 and makes it much less stable across seeds (standard deviation 0.08 against 0.05), which motivated the shift-invariant design. Because this design was adopted during development, we checked that the validation steps alone lead to the same choice: HRAPE has a validation AP of 0.961, against 0.946 for Time2Vec on the same gaps and 0.951 with the

absolute-time encoding (Section 5.4). Against removing the temporal encoding altogether, the gain is smaller (-0.030 for the variant without HRAPE,  $p = 0.26$ ), as expected: all Elliptic edges connect transactions of the same time step (Section 3.2). On the synthetic benchmark, HRAPE is ahead of both alternatives in the no-decoy setting and slightly ahead in the default setting, where the differences are smaller than the standard deviations; in the default setting TGAT is the strongest model (Table 13). The differences are small in the label-noise setting. We have adjusted the claims accordingly.

**Changes in the manuscript.** Sections 3.2, 5.4, 5.9; Tables 6 and 13.

*1.9 The temporal consistency loss in Equation 35 [...] direct supervision on an internal parameter, which looks like label leakage.*

**Response.** We agree and removed this loss. It rewarded high temporal gates on edges incident to fraudulent nodes, i.e. it supervised an internal gate with the training labels. This can leak label information into what is presented as an explanation, and it was not justified. None of the remaining objective terms uses the labels of the explained nodes (Section 3.4).

*1.10 I noticed some inconsistencies in your numbers across different sections [...] make sure all your numbers are consistent and accurately reported.*

**Response.** In the first version the sparsity contribution was reported as 6.37 points in the abstract and 6.47 points in the results, and the HRAPE contribution as 2.53 and 2.51 points. In the revision, no result number is typed by hand. The script `experiments/analyze.py` reads the per-run records and writes every table and a file of LaTeX macros (`tables/numbers.tex`), which the abstract, text, conclusions and this letter all use. A number can therefore not differ between sections. The only intended differences are between studies: the full RTXGNN reaches F1 0.700 in the main comparison (10 seeds), 0.696 in the primary ablation (5 seeds), 0.724 in the secondary ablation (3 seeds, GPU) and 0.718 with all labels in the label-efficiency study (5 seeds, GPU). These studies use different seed sets and hardware, each is compared only with its own reference, and this is now stated in Section 4.4.

*1.11 You reference Figure 4 as showing a visualization of a detected fraud ring, but this figure is not actually present.*

**Response.** We apologise for the missing figure. Figure 5 now shows a laundering ring detected on the synthetic benchmark. Nodes are annotated with their scores, ring transactions (red, solid) are distinguished from ordinary transactions (grey, dashed), and edge widths and labels show SEAL’s edge importance and the transaction day. Because the ground truth is known on this benchmark, we also quantify how well the edge importances identify the ring transactions (edge AUC, Table 14) and compare them with attention weights, GNNExplainer and random scores. Over all detected ring members, the most important incoming transaction is a ring transaction in 91% of the cases. We did not produce such a figure for Elliptic, where no ground truth for rings exists.

**Changes in the manuscript.** Section 5.9, Figure 5, Table 14.

*1.12 The “explainability-as-regularization” framing feels like an overreach [...] standard L1 regularization [...] no comparison against [...] dropout or weight decay.*

**Response.** We agree that the sparsity term is an  $L_1$  penalty on auxiliary gate activations and now say so explicitly (Section 3.4). We ran the requested comparison (Table 7): the mask sparsity term at two strengths, no sparsity term, and no sparsity term with dropout 0, dropout 0.5 or weight decay  $10^{-3}$ . With 3 to 5 seeds per variant, the large effect suggested by the single run of the first version does not hold up; a four times larger sparsity weight even lowers F1. The differences are within the seed variation. We have therefore removed “explainability as regularisation” from the contributions and state only that the explanation objective does not cost detection performance (Section 6.3).

**Changes in the manuscript.** Sections 5.5, 6.3; Table 7.

*1.13 Sub-50ms inference latency [...] What hardware did you use? [...] CPU inference [...] SEFraud claims 0.4ms.*

**Response.** All latency figures were re-measured. Section 5.11 reports new end-to-end measurements on CPU (Intel Xeon Processor @ 2.80GHz, one and four threads). Each request samples the 2-hop neighbourhood from an in-memory CSR graph, builds the tensors, runs the forward pass and extracts the explanation, and every stage is timed separately. We report p50/p95/p99 latency, throughput and peak memory for batch sizes 1, 32 and 256 (Table 16 and Section 5.11). On one core, a single transaction is scored and explained in 2.8 ms at the median and 5.9 ms at the 99th percentile, including neighbourhood sampling, and reading the explanation from the masks takes 0.24 ms. For batches of 256, one core processes 2,013 transactions per second. The model was trained partly on a T4 GPU (described in Section 4.4), but no GPU is needed for inference. Running GNExplainer on the same model instead takes 582.0 ms per transaction. The 0.4 ms of SEFraud refers to a different setting (GPU, amortised over large batches, without neighbourhood retrieval), so the two numbers are not comparable. Our own SEFraud re-implementation is measured under the same protocol as RTXGNN.

**Changes in the manuscript.** Section 5.11, Table 16, Section 6.4.

*1.14 The cross-domain generalization claims are weak. Amazon Computers [...] completely unrelated to fraud detection [...] perfect F1 on synthetic data is trivial [...] A stronger test would be a different real-world financial transaction dataset.*

**Response.** We removed Amazon Computers and added T-Finance (Tang et al., 2022), a real transaction network of a financial company with 39,357 accounts, 21 million edges and expert fraud labels, using the GADBench splits (Section 5.10, Table 15). The synthetic benchmark was redesigned so that it is no longer trivially separable. The first version shifted the features of fraudulent nodes by two standard deviations in every dimension. The new generator contains decoy cycles that differ from laundering rings only in timing, carries no feature signal by default, and has four harder or easier variants. We now describe the claims as tests of transfer to a second financial dataset and of the temporal mechanism under controlled conditions, not as evidence of general cross-domain generalisation.

**Changes in the manuscript.** Sections 4.1, 5.9, 5.10; Tables 13, 15.

*1.15 Typos, grammatical issues, and formatting problems [...] “How ever”.*

**Response.** The manuscript was rewritten and proofread in full, with consistent notation between text, equations and algorithms. The “How ever” artefact came from a line break in the compiled PDF and no longer occurs. We also corrected the duplicated abbreviation entries and several reference entries (for example, the first version cited unrelated papers for Time2Vec and TGAT).

## Reviewer 2

*2.1 The reported improvement over the strongest baseline is modest. Please report results over at least five random seeds with mean  $\pm$  standard deviation and significance testing.*

**Response.** All Elliptic results are now over 10 seeds (5 for the ablations), reported as mean  $\pm$  standard deviation. For every baseline we test the per-seed differences with a two-sided Wilcoxon signed-rank test and apply Holm’s correction over the 16 comparisons (Section 4.4). RTXGNN matches the best GNN baseline and is ahead of the other eleven GNNs on F1, significantly so for several of them before correction and for PC-GNN after correction. On AP, it is significantly better than GCN, GAT, CARE-GNN and PC-GNN after correction. XGBoost is significantly better than RTXGNN. We report all of this in Table 3 and Section 5.1. The same tests applied after an equal-budget hyperparameter search for RTXGNN and ten baselines give the same picture (Table 4).

*2.2 The test split includes steps 35–49, but Table 6 ends at step 48. Please include step 49, class counts per time step, and explain how the overall F1 of 0.5129 is obtained despite near-zero F1 after step 42.*

**Response.** The per-step table now covers all steps 35–49, with the numbers of licit and illicit transactions per step (Table 9). Appendix A gives the counts for all 49 steps. The overall F1 is computed once over all pooled test transactions, not as the mean of the per-step scores. Because 16% of the illicit test transactions occur after step 42, the pooled F1 is dominated by the earlier steps. The mean of the per-step F1 is 0.436 for RTXGNN, against a pooled F1 of 0.700. This is now explained in Sections 4.4 and 5.6.

*2.3 The strong performance collapse after time step 42 is a major practical limitation [...] analyse this drift more carefully and avoid describing the method as production-ready without a retraining or continual-learning strategy.*

**Response.** In the revised runs the collapse starts at step 43, and it affects all models, including the tree ensembles (Figure 2). It coincides with the dark-market shutdown documented by Weber et al. (2019). The number of illicit transactions per step also drops sharply, which makes per-step F1 very noisy. We added a retraining experiment in which the model is refitted before each test step on all steps up to  $t - 5$  and validated on steps  $t - 4$  to  $t - 1$ , i.e. with a one-step label delay. Retraining improves the early steps (RTXGNN 0.83 versus 0.79 static) and recovers part of the late-period loss (RTXGNN 0.10 versus 0.03; random forest 0.18 versus 0.02), but it cannot anticipate a new pattern before labels exist (Section 5.6). The words “production-ready” have been removed, and Section 6.4 describes the monitoring and retraining a deployment would need.

*2.4 The “real-time” claim is not sufficiently supported [...] Please report end-to-end latency, throughput, memory use, and p95/p99 latency, including graph sampling and data-transfer overhead.*

**Response.** Please see our response to comment 1.13. Table 16 reports p50/p95/p99 latency, throughput, per-stage time (neighbourhood sampling including subgraph construction, forward pass, explanation) and peak memory for batch sizes 1, 32 and 256 on one CPU thread, and Section 5.11 adds the results for four threads. Because inference runs on the CPU, there is no host–device transfer. The remaining data-movement overhead (gathering features and edges of the sampled subgraph) is included in the reported times. We replaced “real-time” by “low-latency” in the title and explain that a production system would add the latency of its own graph store.

*2.5 The abstract overstates the improvement. The 19.3% gain is against GAT [...] The improvement over SEFraud is approximately 4.2%.*

**Response.** We agree. The abstract and conclusions now report absolute scores, name the strongest baselines (the best GNN and XGBoost), and report the improvement over SEFraud as an absolute difference (+0.054 F1) with its significance. No relative improvement figure is used.

*2.6 Explanation quality is assessed only through one Fidelity+ value. Please add comparisons with SEFraud and post-hoc explainers, together with stability, random-mask, sparsity-fidelity, and class-wise explanation analyses.*

**Response.** Section 5.8 now evaluates the feature explanations of 600 test transactions (300 illicit, 300 licit) for 5 trained models. For each, we report  $\text{Fid}^+$  (necessity) and  $\text{Fid}^-$  (sufficiency) at  $k = 5, 10, 20, 40$  (sparsity–fidelity curves, Figure 4). The explainers are SEAL’s masks, saliency, Integrated Gradients, GNNExplainer (all post-hoc explainers applied to the same RTXGNN model), a random ranking, and SEFraud’s masks on SEFraud. We also report class-wise results (Table 12), stability under input perturbation, agreement of the masks across independently trained models, and the time per explanation (Table 11). Edge explanations are evaluated against ground truth on the synthetic benchmark (Table 14). In brief: SEAL’s masks are nearly as sufficient as Integrated Gradients at about one hundredth of its cost, more sufficient than saliency (at  $k = 10$  and 20), SEFraud’s masks and random masks, similarly sufficient for both classes, and highly stable under

perturbation. Their necessity is low. GNNExplainer is the most faithful explainer but costs about 300 times more and is unstable. On the synthetic data, the edge importances of the full model identify the planted ring transactions better than GNNExplainer and its attention weights; for the variant without HRAPE, the attention weights are better, and the variation across graphs is large.

*2.7 The regulatory-compliance claims are too strong. Feature masks and anonymous feature IDs may support transparency, but they do not demonstrate GDPR, ECOA, FATF, or MiCA compliance.*

**Response.** We agree and adopted this wording. Please see our response to comment 1.7 and Section 6.2. The regulatory column was also removed from the comparison table (Table 2).

*2.8 The synthetic fraud-ring experiment reporting perfect F1 is not enough [...] The full generation process, seeds, noise settings, class balance, and baseline results should be provided.*

**Response.** The generator was redesigned and is described step by step in Section 4.1 and in the docstring of `rtxgnn/synthetic.py`. The description includes the base graph, background transactions, laundering rings, decoy cycles, features, noise, label noise and split. Table 13 reports five settings (default, no decoys, wide burst, 5% label noise, feature shift) with 5 generated graphs each (seeds 0–4), the number of edges, the positive rate (6.1%), and 13 models including feature-only models, GNNs, temporal GNNs, SEFraud and RTXGNN variants. No model reaches a perfect score in any setting.

*2.9 The label-efficiency results are unusual because performance at 50% labels is lower than at 20%. Please repeat this experiment across multiple subsets and seeds, and report uncertainty.*

**Response.** Please see our response to comment 1.6. The experiment is repeated with 5 stratified subsets per fraction, and Table 10 and Figure 3 show means and standard deviations. With repeated stratified subsets, RTXGNN’s F1 increases monotonically with the label fraction (from 0.593 at 5% to 0.718 at 100%). The random forest and GAT still show non-monotonic means, but their differences between neighbouring fractions are within one standard deviation across subsets. This indicates that the non-monotonic value of the first version reflected the variance of a single random subset.

*2.10 The manuscript says nine baselines but lists more; the fraud-specific baseline subsection is repeated; Table 4 says No-SEAL recall is omitted although it is reported; and Algorithm 1 appears after the references.*

**Response.** Thank you. All four issues are corrected: the baseline list is accurate (16 baselines, Section 4.3), there is no duplicated subsection, the ablation table was regenerated from the run records with a correct caption, and the algorithms are placed in the method section.

*2.11 The case-study interpretations are speculative because Elliptic features are anonymized. Please avoid assigning real-world meanings to feature IDs.*

**Response.** We agree. The case studies (Section 5.12, Table 17) now report only the feature index and its documented group (local or aggregated), the number and composition of the open features, the effect on the score of removing the leading five features, and the most important neighbouring transactions with their labels. Statements such as “likely unusual amounts, timing patterns” and the hypothetical mapping “Feature 51 = transaction velocity in last 24h” were removed.

We thank the reviewers again. Their comments have substantially improved the method, the evaluation and the presentation, and we hope that the revised manuscript addresses all of their concerns.

Sincerely,  
The authors